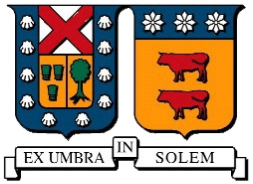
ELO320 - Estructuras de Datos y Algoritmos
Introducción a C
Struct y Strings

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

2do Semestre 2022



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Estructuras

- Conjunto heterogéneo de elementos de datos.
- Variable: campo, miembro o atributo.
- Base de los Tipos de Datos Abstractos (TDA).
- Encapsulamiento de información.
- Acceso: Operador punto (.).

C

```c
struct empleado {
    char nombre[40];
    float sueldo;
};

int main(){
        struct empleado e1;
        e1.sueldo = 550000.00; //acceso a sueldo
        strcpy(e1.nombre, "Juan"); //acceso a nombre
        printf("El empleado %s gana %.2f USD al año", e1.nombre, e1.sueldo);
        return 0;
}
```

Estructuras y punteros.

- struct → nuevo tipo de datos → punteros.
- El operador punto se combina con derreferencia, e.g., `(*var).nombre`;
- Ojo, `(*var).nombre` ≠ `*var.nombre`.
- En vez, se puede usar el operador `->`.

C

```
struct empleado {
    char nombre[40];
    float sueldo;
};

int main(){
        struct empleado e1;
        struct empleado *ptr;
        ptr = &e1;
        ptr->sueldo = 550000.00; //acceso a sueldo
        strcpy(ptr->nombre, "Juan"); //acceso a nombre
        printf("El empleado %s gana %.2f USD al año", ptr->nombre, ptr->sueldo);
        return 0;
}
```

Definiendo nuevos tipos de datos

- **typedef** → crear nuevos tipos de datos.
- Se utiliza para encapsular structs.
- También para no trabajar con tipos de datos comunes de forma directa.

C

```c
typedef float real;

typedef struct empleado {
    char nombre[40];
    real sueldo;
} empleado_t;

int main(){
        empleado_t e1;
        e1.sueldo = 550000.00; //acceso a sueldo
        strcpy(e1.nombre, "Juan"); //acceso a nombre
        printf("El empleado %s gana %.2f USD al año", e1.nombre, e1.sueldo);
        return 0;
}
```

Strings

- Estructura de Datos para poder representar palabras.
- Trabajan sobre arreglos de caracteres.
- La norma es que finalizen con el caracter fin de string: '\0'.
- **string.h** → funciones.
- Dependen de la existencia del caracter '\0'.
- Indexados como todo arreglo.
- Comillas (") → string, apóstrofe (') → char.
- TDA String: funciones + arreglo de caracteres + '\0'.

Index	0	1	2	3	4	5
Variable	H	e	l	l	o	\0
Address	0x23451	0x23452	0x23453	0x23454	0x23455	0x23456

Definición

C

```c
int main(){

        char a[10]; //Tamaño fijo.
        char *b; // Tamaño dinámico.


        a="Hola"; // inválido
        b="Hola"; // válido

        return 0;
}
```

Inicialización

C

```
int main(){

        char a[10]="Hola"; //Tamaño fijo con valor inicial.
        char a2[10]={'H','o','l','a','\0'}; //obligación el '\0'

        char b[5]="Juan"; // Éstas dos son equivalentes.
        char b2[]="Juan" // Tamaño fijo definido con valor inicial.

        char *c = "ELO320"; // String constante, tamaño variable.

        char *d = malloc(...); // String dinámico, puntero.
        strcpy(d, "String Pro");
        free(d); //próx. clase.

        a[0]='D'; //válido
        b[0]='D'; //válido
        c[0]='D'; //no valido, segmentation fault.
        d[0]='D'; //valido, por algo es pro.

        return 0;
}
```

Biblioteca string.h

- `#include<string.h>`
- Contiene funciones y elementos útiles para manejar strings.

C

```
int main(){

        char a[10];
        char *b, *c;
        int tam;

        strcpy(a,"Hola"); // strcpy(a,b) copia en a lo de b
        //strncpy(a,b,n) copia los n primeros caracteres.
        //retornan un puntero a "a".

        b=" Mundo";

        c=strcat(a,b); // concatena a con b;
        //strncat concatena a "a" los primeros n caracteres de b.
        //retornan un puntero a "a".

        tam=strlen(a); //largo de a. no cuenta '\0'.
        // retorna size_t, es un typedef de unsigned long int.

        printf("tamaño: %d\n", tam);

        return 0;
}
```

Biblioteca string.h

Mas info:
`https://en.wikibooks.org/wiki/C_Programming/string.h`

Desafíos

- Escriba un programa que pide al usuario un número entre 1 y 100, y a continuación llama a una función que devuelve la suma de todos los números pares que van del 1 al número que dio el usuario. Al final esta suma se imprime en la pantalla.
- Escriba un programa que, dada la hora en su computadora, llama a una función que calcula y devuelve el ángulo entre las manecillas del reloj (hora(s) y minutos). Este valor el programa lo imprime en la pantalla. (miren en los códigos de ayuda sobre cómo usar la librería time.h para acceder a la hora en su sistema).
- Escriba un programa que, dado un arreglo de enteros de dos dimensiones (array [][]) inicializado de manera random (fíjense en las funciones en stdlib.h), definido global, desarrolle un algoritmo que encuentra el mínimo valor en el arreglo entero pero pasando una sola vez por cada elemento del arreglo.